

YAML

```
name: webdev
description: Development workflow, best practices, and platform references for
```

Webdev

Workflow

The plan records how to build the product; the ToDo records what was promised. Product requirements describe what the application's users can do, not a script for the development agent to perform. Build the product using code inspection, automatic diagnostics, and existing

tests as normal development feedback. A Web project that entered Plan Mode requires one independent review of how its application parts connect before delivery. Reuse that planning decision rather than

judging the project's complexity again. For projects that did not enter Plan Mode, an extra acceptance pass remains optional; when its value is unclear, finish without an extra pass.

Screenshots and browser interaction are off by default for development validation. Use them

only when the user explicitly requests that method or an already observed visual/interaction

defect requires it. A desired feature, complexity, or missing evidence is not that exception. A method added

by the agent to a plan or ToDo is not a user request; revise speculative checks instead of executing them. Read-only operation and tool/reference suggestions do not change this boundary.

Prefer the validator's code-based review over screenshots or browser interaction. When permitted

evidence requires seeing the rendered result, prefer `webdev.take_screenshot` ; use browser interaction only when the specific question requires operating the page. The screenshot tool

description owns capture behavior and image inspection.

After attach, continue the current request using the restored project and existing plans; do not repeat initialization or initial planning unless the scope changes. The success receipt includes this Skill and selected saved configuration. Reuse supplied facts for discovery; read config for missing fields, writes, recovery or concurrent changes. Project-file locators

are

relative to the returned project directory; documentation links are relative to this Skill's directory. A saved port is not evidence of a running service.

Tool parallelism

DEFAULT TO PARALLEL TOOL CALLS: batch every ready read, write, edit, exec and MCP call

within the host limit, including different tools and repeated calls to the same tool.

Emit operations in their intended order. Web domain PUTs replace only that domain; secret PUTs change only that key. Independent updates do not require separate model turns or a shared

project revision. Whole-document PUT replaces the document. Order overlapping changes and

real dependencies; wait when you need to inspect a result, obtain user input/confirmation, or verify required runtime synchronization. Keep explicitly standalone tools alone.

A pending secret card with `blocks_config_writes: false` allows unrelated domain updates; conflicting keys, dependent changes and whole-document replacement must wait or use `unblock`.

1. Plan

Required reading before a Web project plan: Before drafting or writing `plan.md`, read [webdev-deployment](#) in full, unless its complete contents are already in the current context. Complete this step during planning, even when the framework or deployment

approach seems obvious or has already been chosen. Use the guidance to specify frontend/backend

serving, production routes and cache policies in the plan; do not defer these decisions or this

reading until implementation. For public pages intended for search, also read [webdev-seo](#) during planning.

The plan explains how to implement the requested product and how its parts fit together. Keep

the agreed product outcomes and the decisions that affect them, so later work does not silently

change the scope. Describe the implementation approach, structure, dependencies, and material

constraints. Preserve concrete values or wording when they affect the product result, and keep

required behavior distinct from optional ideas.

The plan must be grounded in the project's actual capabilities and constraints. Before writing

`plan.md`, identify which parts of the requested product depend on platform or template guidance.

Use the Skill reference descriptions to select the relevant guides and read them in one parallel batch, and inspect project

files with read-only tools to resolve the decisions the plan depends on. Complete this necessary

reading during Plan Mode, before committing to the approach; do not defer it to implementation.

Reuse guidance already supplied by initialization or earlier reads while it remains in context; there is no need to read the entire reference catalog. If later reading changes a planning assumption, update the plan before implementing the affected work.

For ordinary delivery, the verification section describes how code inspection, automatic diagnostics, and existing checks establish the requested outcomes. It does not need a list of new tests. Leave screenshots, browser sessions, and newly authored acceptance scripts out of

the plan and `ToDo` unless the user requests those methods or a concrete observed defect needs

them. Do not turn the application's user journeys into a testing itinerary. A generic request to test and deliver, or approval of an agent-authored plan, does not add that assignment.

2. `ToDo`

The native `todo` carries the acceptance specification from planning to implementation and independent review. A reviewer must be able to judge an item from its text; a module name or

summary of the plan does not preserve the promise.

After plan approval, create this list before configuration or implementation work. Read the plan if needed. Replace preliminary phase headings using `todo` `action=create`; retain real progress only where the same outcome has already been delivered.

Construct the list from the actual acceptance clauses in the user request and approved plan. When the request enumerates product features, make an item for each feature group and carry

that group's acceptance clauses into it. Merge duplicate clauses, not distinct feature groups.

For unstructured requests, identify the independently deliverable outcomes before grouping. Copy clear source clauses into their outcome items rather than paraphrasing or summarizing them during conversion. Preserve the exact values, messages, field rules, actors, state transitions and exceptions. Where the plan omitted or weakened a

requirement,

use the original request. Exclude secret API keys and implementation rationale.

Store an outcome heading and these source clauses together in `title`, the item's only text field. The heading can be short; the acceptance specification can be a full paragraph or several related clauses. Do not optimize it into a one-line display label. “All rules”, “as specified”, “correct values” and links to the plan are not substitutes for the clauses.

Group clauses that describe the same independently deliverable outcome. Do not bundle a whole application into one item, impose a tiny item count, or split every field into its own task. Before calling `todo`, check the copied clauses against the source: every required condition must still appear in the list, with duplicates consolidated rather than generalized.

Acceptance clauses describe product behavior, not additional tests. Reuse code and existing evidence. Preserve IDs and full criterion text in subsequent status updates; mark an item completed only when evidence establishes its outcome, and keep unresolved outcomes open.

For a Web project that entered Plan Mode, also track the required independent review in the `ToDo`. Record its findings against the affected product items and keep unresolved outcomes open.

3. Implement

Use the returned project directory and setup state, and discover the current project tools with `tool.list` on `webdev-mcp`. Follow the selected application kind's implementation guide from the Skill reference below. The following setup steps apply to Web projects.

Before application code, read [design](#) and follow its design-direction, `ideas.md`, required project branding, and asset workflow. Reuse the approved Blueprint or design direction. Apply the [development workflow](#) to setup that is still needed; an initialized starter does not need to be scaffolded or installed again.

Before relying on edit diagnostics, follow the applicable host-managed or agent-managed flow in [development](#). Host-managed Web/mobile starter projects register diagnostics during initialization; a ready application alone is not proof of checking.

Before the first Web server start, prepare the complete current-source [website route manifest](#), in both Cloud and Studio.

Keep the route manifest synchronized as routes change.

Read the capability guidance needed for the current work, and provide the infrastructure the

requested product needs; split delivery into stages only when the user requested that scope.

Implement against the plan and `ToDo`, reusing automatic diagnostics and existing results.

4. Validate before delivery

If this Web project entered Plan Mode, run one read-only independent validator before delivery. Blueprint's `needs_plan` route already made the complexity decision; do not make a second discretionary decision about whether this review is worthwhile. An approved

implementation plan from Plan Mode is sufficient to require it. A compact Blueprint card alone is not Plan Mode.

For a project that did not enter Plan Mode, choose additional review only when remaining uncertainty warrants it. Reuse passing results and automatic diagnostics. Ordinary stacks do

not need a local production, clean-checkout or Docker build before Publish.

Read [Validator](#) before composing its task. Pass the approved plan path, complete `ToDo` IDs and criterion text, project directory, existing evidence and allowed checks. The validator checks how the implemented parts connect, starting with code and existing

evidence. Its guide owns scope selection, review methods, results and follow-up. Fix confirmed

findings and keep unresolved outcomes open in the corresponding `ToDo`; do not claim unchecked

requirements passed.

Best practices

- Users can see the live Preview in the Dashboard widget as you update the code, making development more visible and easier to follow. Once you have enough of the interface to meaningfully show the user, start the development server on the Preview port so they can see the product taking shape.
- Include a project structure section in the plan, describing the main folders, modules, and their responsibilities.
- Prefer a well-defined project structure over writing a very large amount of code in a single file.
- Prefer strict, explicit type definitions so errors can be caught and corrected early during development.

- Keep initial loads small: load what the current page needs and defer substantial dependencies for optional pages or features until use. Use the framework's existing code-splitting and lazy-loading support.
- For good user-perceived performance and experience, usually prefer static frontend delivery with backend APIs when it fits the application. SEO, first-page content, personalization, freshness, framework constraints or an explicit user preference for SSR may call for a different approach. Use [webdev-deployment](#) to choose the architecture, routing and caching that best meet the requirements.
- Usually cache versioned public assets for a long time, keep HTML able to pick up new releases, and keep private responses out of shared caches. Adapt each policy to the content's freshness and privacy requirements; the deployment skill gives the concrete choices.
- 确保前后端数据契约一致，涵盖类型、格式、可空性及序列化行为；交付前用真实 API 响应
对实际前端解析逻辑进行自动化验证。
- Prefer optimistic UI updates for ordinary interactions: reflect the user's action immediately while saving in the background, reconcile with the server response, and roll back with a clear error if saving fails. For sensitive operations such as financial transactions, wait for server confirmation before showing success.
- For container deployments, Publish runs the Dockerfile on the platform's build machines and returns build errors. Include the required compilation steps for what the container serves, and start the production application.
Skip an extra sandbox build unless there is a concrete need; it adds time and consumes sandbox resources.

Cloud versus Studio

For Local website or mobile app requests, read [webdev-worklocally](#) for rediscovery and managed-versus-plain-local choice before entering this Addon workflow. For Mobile, resolve that choice before switching to Cloud; the Local guide owns the disclosure and routing.

A found declaration, Project website or suggested Resource is context, not an instruction to

attach or turn unrelated work into a website. Follow the user's choice instead of initializing or pushing a plain-local website. Complete the existing directory/authorization steps before binding; the Local guide owns target clarification, file identity and independent Managed Git.

Environment	Working environment and Preview	Detailed guidance
Cloud	Project files live on Sandbox local disk; frontend file views synchronize asynchronously. Save Git checkpoints for recovery after Sandbox replacement.	Cloud workspace, runtime
Manus Studio	Work runs in the authorized local worktree, using the owning Desktop's environment and permissions.	Local environment and Preview

Checkpoint

A checkpoint is a saved version of the website, identified by an accepted canonical `main` commit and displayed as a checkpoint card and in Version history. Cloud records it after an accepted canonical-main push. Studio uses its separate Manus Git operation, preserving the user's own Git configuration. A local commit alone does not record a platform checkpoint. The [checkpoint guide](#) covers synchronization and returned status.

Save or connect this website to GitHub

A request to save this managed website's code to the user's own GitHub is a Webdev repository switch, unless they explicitly want a separate backup without changing the managed repository.

Read [Git canonical](#) before choosing tools. Use `webdev.config` to prepare, then transfer through its temporary remote and complete the switch. Do not use a GitHub connector, `gh repo create`, or a replacement `origin` to bypass this flow. Keep requested branches separate; a GitHub push alone is not confirmation that Webdev has switched repositories.

Preview

Preview serves the current application from its running development process. In Cloud it is proxied into a cross-site iframe; in Studio it opens the Session's local listener. Its address and port come from the runtime receipt. Preview is a development view, not a saved version or a permanent public deployment. Listener and proxy details are in [runtime](#); Studio's listener and access rules are in [Local](#).

Publish

Publish builds a permanent deployment from a saved checkpoint, using a clean checkout of that commit. Local dependency folders and scratch files are not build inputs. The platform returns publication status and the public address; a successful config write, Preview, or checkpoint alone does not establish publication success. The mode-specific publication flow and receipts are described in [Git/checkpoints](#).

There are three publication entries: Git push auto-publish when `GET config` reports `publishing.auto_publish: true`, an authorized MCP `POST publish`, and the user's Dashboard Publish action. MCP publish runs as a background job and automatically returns its final result; continue other work or end the turn without polling. Git auto-publish and Dashboard publish show completion in the Dashboard without sending a completion message to the agent. Never enable auto-publish yourself. Authorization, entry selection, and result handling are defined in [Git/checkpoints](#).

Configuration and capability choices

`webdev.config` operates the bound project's control plane. Application code uses the runtime APIs, not this tool or its control-plane credentials. Configuration is stored on the platform, not in an editable project config file. Read and write semantics, responses, and interactive confirmation state: [configuration](#).

Protected input is collected by `webdev.request_secrets`; its tool description owns the invocation and waiting behavior. Declaration and revision contracts: [secrets](#).

`features.server` defaults to false; enabling managed features is one-way. A server changes the deployment shape, and managed database access requires it. The feature declaration contract is in [features](#). Capability-specific prerequisites are also discoverable below; a business requirement does not itself select a platform provider.

The platform provides the following services for your application. Use this table to identify capabilities that fit the requested product, then read the linked guide for setup and integration details.

Capability	What it provides and when to use it
Database	A managed MySQL-compatible database for persistent application data, such as user profiles, orders, uploaded-data records, and saved content. Use it when data must survive page refreshes, application restarts, and new deployments. Requires a server-capable application. See database .
Login	Manus OAuth is the default when adding login, sign-up, user accounts, or authentication without a specified provider. Read authentication before implementing login; it owns provider selection, application sessions, and permissions. Requires a server-capable application.
Payments	A managed Stripe integration for one-time purchases and subscriptions, including test-account provisioning, payment credentials, and webhook registration. Use it when the application needs checkout and payment-driven behavior, such as granting access after payment. Accepting real payments requires the owner's live-account setup. See payments .
Shopify commerce	Create a new Shopify store or connect an existing one, then build a storefront with products, collections, a cart, and Shopify checkout. Use it for Shopify-backed online stores. The platform supplies Storefront access and provides development-store administration tools while the store is eligible. See Shopify .
File storage	Durable storage for uploaded files, images, audio, and generated documents, with stable addresses that the application can display or link to. Use it when files need to remain available after a development session or deployment ends, including user uploads

	and generated assets saved for later use. See storage .
Runtime service access	Platform-provided API addresses and project credentials for calling AI and utility services from application code. Use this shared access mechanism when integrating the LLM, image, speech, storage, API hub, or notification services below. The guide explains server authentication and the browser access supported by individual services. See service API .
LLM inference	Built-in language-model access for conversation, text generation, summarization, translation, extraction, and analysis of application inputs. Supports streaming and model-dependent structured or multimodal inputs and outputs. Use it when the application needs model-generated responses or content. No separate model-provider account or API key is required; usage deducts from the project's credit balance. See LLM .
Image generation and editing	Generate an image from a text description or edit supplied images through an application runtime API. Use it for features where users create or transform images inside the application. Results that users need to keep can be saved to project storage. See images .
Speech transcription	Convert uploaded or recorded audio into text, with detected language and timestamped segments. Use it for voice input, recording transcripts, searchable audio content, or subtitle workflows. See speech .
Maps and places	Google Maps access using platform credentials, including interactive and static maps, address lookup, place search and details, autocomplete, directions, and distance calculations. Use it for location search, store finders, travel planning, or other location-based features without asking the user for a separate Google Maps key. See maps .
External data APIs	Access to Manus-managed external data APIs without separate provider credentials. Use it

	when the application needs to search, retrieve, or display information from supported external services—for example, the documented <code>Youtube/search</code> API. Available integrations depend on the registered catalog; use API search when the session exposes it to find a suitable integration before arranging separate provider access. See API hub.
Owner notifications	Send a notification with a title and message to the Manus project owner. Use it to alert the owner about application events, completed work, or problems needing attention. The recipient is fixed to the project owner; customer messaging needs a separate service. See owner notifications.
Scheduled work	Run application work on a recurring schedule by having the platform call an endpoint on the published site. Use it for periodic data refreshes, report generation, maintenance, or recurring checks that must continue independently of an application process. Schedules and run history can be managed through the platform. See scheduled work.
Installable web app manifest	For eligible published sites with a project logo, the platform can generate a web app manifest using the project's title and logo, with standalone display when installed. Use the default for basic manifest needs, or let the application supply its own manifest for custom installation behavior. Offline support and service workers require application implementation. See PWA.
Website deployment and performance	Choose static, SPA/API or SSR serving, build contracts, published routing and response caching during planning or delivery work. See webdev-deployment.
Search discoverability	Implement indexable public pages, metadata, canonical URLs, sitemap and robots behavior. See webdev-seo.
Hosting resources and idle behavior	Choose the published application's CPU and memory tier and whether it may sleep when idle or should remain resident. Use these settings when the application needs more

	resources or when cold-start delay matters. Some options require owner membership. See hosting .
Diagnostics and logs	Sandbox manages LSP for registered project directories; inspect or select languages through config. On a local Device, you manage native servers and settings. See diagnostics for setup/routing/settings and published-site logs, and the error index .

Skill reference

Decide which guides the current step needs, then read them together in one parallel tool call instead of one at a time; guides already in context need no re-read.

Reference	When to read it and what it covers
Design	Before application code: design direction, <code>ideas.md</code> , and visual assets.
Development	When preparing dependencies, starting the development service, or selecting development checks.
Default template	When the initialized project uses <code>web-db-user</code> : supplied code, completed setup, commands, and later changes.
Route manifest	Before the first Web server start and when routes change: declare the complete current route set.
Cloud workspace	When working with the remote project filesystem, project state, or workspace recovery.
Local environment	When working in Studio: authorized directories, local dependencies, managed environment, Session Preview, and recovery.
Runtime	When configuring or diagnosing the development listener, Preview proxy, or language diagnostics.

Next.js	Before configuring a Next.js development server, or when Preview renders but controls fail or HMR repeatedly reconnects.
Validator	Before delegating an independent validator: scope, inputs, connection checks, results and follow-up.
Git/checkpoints	When saving a website version, synchronizing Manus Git, publishing, or interpreting checkpoint receipts.
Git canonical	Before saving/connecting this website to the user's GitHub, returning it to Manus, or continuing/canceling a repository transfer.
Website deployment and performance	Before deployment architecture decisions, publication setup, or investigation of slow loads, stale assets and broken direct links.
SEO	For public pages intended for search, SEO implementation, or indexing problems.
Game setup	Read before new Game init. After init/attach, follow the receipt for the recorded <code>gameEngine : threejs</code> uses its own starter README and the Web development, build and checkpoint flow; <code>godot</code> uses the delivered Godot workflow . After context loss recover the same engine's guidance; Three.js does not read the Godot workflow or delivery guide. Online features use only the selected guides from Game Blueprint .
Game sharing	For either engine, before sharing metadata/OG work or <code>game/sharing</code> writes.
Mobile	For an initialized Mobile project: use its fixed starter, device/backend, development, and app-build workflow.

Delivery

For an ordinary website handoff, keep the final reply within 100 words unless the user requests

detail. State the result and blockers plainly and deliver the website with a single URL: use the published URL only when a publication result confirms that the version being

delivered

is live; an earlier version's successful publish does not establish this. Otherwise provide the current Preview URL, clearly identified as a preview. If no current URL is available, state that limitation rather than inventing one.

The website itself needs no file attachments. Do not attach source, README/plan files, process screenshots, or internal assets as website deliverables. Provide files only when the user requests them or a separate file is itself a requested deliverable.

Refer to an existing checkpoint by its SHA and card/Version history; do not invent a resource URI or closing tool. Preview/checkpoint success is not publication success.

Project directory in Studio and Cloud Computers

For Local First Web/Game init or attach, optional `project_dir` chooses an existing absolute project directory accessible to the Host, including outside the task directory while leaving the task's initial directory unchanged. If starting in Home/Documents, create a dedicated child folder and pass its absolute path; do not ask the user to switch workspaces for this.

Omit the parameter when the current task directory is already the intended project.

Ordinary platform-managed Sandbox allocates its own directory, so omit it there. Follow `worklocally/SKILL.md` for setup choices. After init/attach, use the returned `project_dir` for file paths and terminal `cwd` ; Webdev config, preview and Managed Git follow that project.